

Google File System (GFS): Project Implementation Plan

Team 7: Import Dependency

1 Abstract

In this project, we aim to implement a small replica of one of the key contributions to the distributed systems community: the Google File System (GFS), based on the seminal 2003 paper. This implementation will include key MVP functionalities such as Read, Write, Record Append, Stale Replica Handling, Garbage Collection, Operation Logging, and Checkpointing. Furthermore, we will benchmark the performance of our GFS replica across various aspects in granular detail to evaluate its scalability and fault tolerance for large, data-intensive applications.

2 Architecture Components

2.1 Master Node

The Master is the central coordinating node. It stores the mapping from files to chunks, the location of chunk replicas on Chunk Servers, and file namespaces (the hierarchical directory structure). To ensure fast lookups and high performance, all metadata is stored entirely in memory.

2.2 Chunk Server

Chunk Servers store the actual data chunks and their replicas. They send regular heartbeats to the Master node to report their status and confirm they are alive. As described in the original paper, the chunk size is fixed at 64MB. All data-bearing operations (reads/writes) bypass the Master; clients connect directly to Chunk Servers to perform operations after fetching metadata.

2.3 Clients

Clients are responsible for initiating all operations, including retrieving file info, creating directories/files, reading, and writing. The client caches the location of Chunk Servers (chunk handles and replica locations) for a limited time to facilitate fast subsequent access and drastically reduce the load on the Master node.

3 Core Operations

Below are the detailed workflows for the core data and metadata operations.

3.1 Read Operation

1. **Calculate Index:** The Client translates the application's filename and byte offset into a chunk index.
2. **Metadata Request:** The Client sends the filename and chunk index to the Master.
3. **Metadata Response:** The Master replies with the chunk handle and the network locations of all current replicas.
4. **Caching:** The Client caches this metadata using the filename and chunk index as the key.
5. **Data Request:** The Client sends a request to one

of the replicas, typically prioritizing the closest one.

6. **Data Transfer:** The chosen Chunk Server reads the data and returns it to the Client.

3.2 Write Operation

1. **Lease Request:** The Client asks the Master which Chunk Server holds the current lease for the chunk (the "Primary") and the locations of the "Secondaries".
2. **Data Push:** The Client pushes data to all replicas. Replicas store this data in an internal LRU buffer.
3. **Write Request:** Once all replicas acknowledge receiving the data, the Client sends a write request to the Primary.
4. **Serialization:** The Primary assigns consecutive serial numbers to all incoming mutations and applies them locally.
5. **Forwarding:** The Primary forwards the write request to all Secondaries, which apply mutations in the same order.
6. **Acknowledge:** Secondaries reply to the Primary.
7. **Client Reply:** The Primary replies to the Client.
8. **Chunk Boundary Handling:** If a write exceeds 64MB, the Client writes what fits, then requests a new chunk handle for the remainder.

3.3 Record Append Operation

1. **Initialization:** The initial steps are identical to the standard Write operation. The Client asks the Master for the leaseholder, pushes the data to all replicas' LRU buffers, and sends the append request to the Primary. However, the Client does *not* specify a byte offset.
2. **Boundary Check:** The Primary checks if appending the record exceeds the 64MB chunk boundary.
3. **Padding:** If the record exceeds the boundary, the Primary pads the current chunk to its maximum size, instructs Secondaries to do the same, and returns an error instructing the Client to retry the operation on the next chunk.
4. **Appending:** If the record fits, the Primary appends the data to its replica, forwards the specific offset to Secondaries so they write at the exact same location, and replies success.

3.4 Memory Management & GC

1. **Lazy Deletion:** Master logs deleted files and renames them to a hidden, timestamped name without immediate deletion.
2. **Background Scan:** A periodic task removes hidden files older than a set interval, freeing Master memory.
3. **Orphan Removal:** During heartbeats, the Master identifies metadata-less chunks and orders their physical deletion.

3.5 Stale Replica Handling

1. **Version Numbering:** Master increments a Chunk Version Number every time a new lease is granted.
2. **Missed Updates:** Disconnected Chunk Servers miss mutations and version updates.
3. **Detection:** Upon reconnection, the Master notices the outdated version number.
4. **Isolation:** The Master ignores the stale replica and schedules it for garbage collection.

3.6 Operation Logging & Checkpoints

1. **Logging:** Master logs historical metadata mutations to an Operation Log before acknowledging the Client.
2. **Checkpointing:** Master creates a compact B-tree checkpoint of its state when the log hits a size limit.
3. **Truncation:** The log truncates post-checkpoint. On restart, the Master replays only logs after the latest checkpoint.

4 Implementation Strategy

- **Language:** Python 3.9 (Master, Chunk Server, Client).
- **Framework:** gRPC with protobuf serialization.
- **Storage:** Local OS file system on Chunk Servers.

5 Tentative Timeline

The project will be executed over a four-week timeline, compartmentalizing the architecture into distinct deliverables.

5.1 Week 1: Protocol Design

Focus on defining the communication backbone and data schemas.

- **gfs.proto:** Design the gRPC service definitions. Implement `MasterService` (methods like `GetFileInfo`, `GetChunkLocation`, `Heartbeat`) and `ChunkServerService` (methods like `ReadChunk`, `WriteChunk`, `PushData`). Define core message structures including `ChunkHandle`, `FileInfo`, and `Mutation`.

5.2 Week 2: Master Node

Focus on the centralized control plane and metadata management.

- **master.py:** Build the in-memory data structures mapping file namespaces to chunk handles, and chunk handles to replica locations. Implement lease allocation logic, chunk versioning, and the background thread for Garbage Collection and stale replica detection based on incoming heartbeats.

5.3 Week 3: Chunk Server

Focus on the data plane, storage mechanisms, and replication logic.

- **chunkserver.py:** Implement local OS file system I/O to map `ChunkHandles` to physical files. Build the LRU buffer for decoupling data flow from control flow. Implement the serialization of mutations on the Primary replica and the forwarding logic to Secondaries to maintain consistency.

5.4 Week 4: Client Logic & Integration

Focus on user-facing operations, caching, and end-to-end testing.

- **client.py:** Build the metadata cache to store chunk locations. Implement the complex Write and Record Append workflows (coordinating data pushes to replicas and control requests to the Primary). Conduct full integration testing, handling edge cases like chunk boundaries.

6 Future Enhancements

If time permits, we plan to extend the MVP to include advanced features detailed in the GFS paper and modern distributed systems:

- **High Availability (RAFT):** The current MVP relies on a single Master, creating a single point of failure. We plan to implement the RAFT consensus algorithm to deploy a multi-master cluster, ensuring automated leader election and robust fault tolerance.
- **Global Consistent Snapshots:** Implement the copy-on-write snapshot feature described in the paper. This allows for instantaneous, zero-copy backups of massive directory trees by duplicating metadata and only copying physical data blocks when subsequent mutations occur.
- **Data Integrity via Checksums:** Implement 32-bit checksums for every 64KB block within a chunk. Chunk Servers will verify these checksums on reads to detect data corruption early.

7 Benchmarking

To strictly evaluate the system's performance, we will conduct quantitative micro-benchmarks reflecting the methodologies in Section 6 of the GFS paper:

- **Aggregate Read Throughput:** Measure sustained MB/s across N concurrent clients reading distinct 64MB chunks, identifying network saturation limits versus disk I/O bottlenecks.
- **Aggregate Write Throughput:** Measure MB/s for concurrent clients writing to distinct files, evaluating the pipeline efficiency of the data push and control flow separation.
- **Record Append Concurrency:** Benchmark the throughput and latency of multiple clients appending heavily to a single, shared file to stress-test the Primary's serialization and padding logic.
- **File Size Impact on Latency:** Measure how varying file sizes (from small KB files to multi-GB datasets) impact end-to-end read/write latency. This will highlight the proportional overhead of Master metadata lookups for small files versus bulk data transfer speeds for large files.
- **Chunk Boundary Overhead:** Evaluate the latency penalty incurred when operations cross the 64MB chunk boundary, requiring the Client to fetch new metadata from the Master and establish connections with a different set of Chunk Servers.
- **Master Operation Latency:** Measure the ops/sec for metadata lookups and lease grants to ensure the Master does not become a system-wide bottleneck.